DOC RMT-ARCH-2026-001
July 2026

TECHNICAL DEVELOPMENT PLAN & ARCHITECTURE BLUEPRINT

B2B University Aggregator & Admission Platform

A comprehensive engineering blueprint for the **RIMIT Education & SPES Education** unified B2B portal — consolidating India's university ecosystem into a single cloud-native platform with centralized admission management, sub-center partner workflows, and omnichannel notifications.

PREPARED FOR
RIMIT Educational Charitable Trust

POINT OF CONTACT
Abdul Bari Chengana, MD

STACK
Django · PostgreSQL · Keycloak · Celery

TIMELINE
20 Weeks (5 Phases)

CONFIDENTIAL | MODULAR MONOLITH | RLS MULTI-TENANCY | OIDC / MFA

1. Executive Technical Strategy **4**

2. RFP Deconstruction & Functional Mapping **6**

2.1 Functional Module → Sub-System Mapping 6

2.2 Non-Functional Requirement (NFR) Extraction 6

2.3 Regulatory & Compliance Alignment 7

3. Architectural Visualizations **8**

3.1 C4 Level 1 — System Context Diagram 8

3.2 C4 Level 2 — Container Architecture Diagram 8

3.3 Cloud Infrastructure & Deployment Topology 9

3.4 Critical Transaction Sequence — Student Enrollment 9

3.5 Draw.io Visual Styling Specification 10

4. Component Specification & Technology Evaluation Matrix **11**

4.1 Ingress / Edge Routing 11

4.2 Compute / Application Layer 11

4.3 Data Persistence (OLTP) 12

4.4 Caching & Asynchronous Messaging 12

4.5 Identity & Access Management (IAM) 13

4.6 Object Storage / Document Vault 13

5. Database Schema Walkthrough & Multi-Tenancy **15**

5.1 Module 1 — Aggregator Hub Tables 15

5.2 Module 2 — Partner & User Governance 15

5.3 Module 3 — Student Registration & Academics 15

5.4 Module 4 — Admissions & Finance 16

5.5 Module 5 — Compliance & Observability 16

5.6 Multi-Tenancy via Row-Level Security (RLS) 16

6. REST API Specifications by Module **18**

6.1 Module 1 — Aggregator Hub Endpoints 18

6.2 Module 2 — B2B Sub-Center Portal Endpoints 18

6.3 Module 3 — Business Logic & Rules Endpoints 19

6.4 Module 4 — Marketing & Communication Endpoints . . . 19
6.5 OpenAPI Documentation & SDK Generation . . . 19

7. UI/UX Screen Inventory & Design System 20

7.1 Screen Inventory by Module . . . 20
7.2 Design System & Tokens . . . 20
7.3 Workflow Governance & Revision Cap . . . 21
7.4 Performance & Accessibility Rules . . . 21

8. External Integrations Design 22

8.1 Meta Lead Ads Webhook Integration . . . 22
8.2 WhatsApp Business API Integration . . . 22
8.3 Payment Gateway — Razorpay Integration . . . 23
8.4 SMTP / Email Notifications . . . 23
8.5 Document Vault — MinIO/S3 Integration . . . 23

9. Security, Governance & Compliance Posture 24

9.1 Identity Control & Token Flow . . . 24
9.2 Data Protection Vectors . . . 24
9.3 Compliance Alignment Matrix . . . 24
9.4 Audit Trail & Forensic Readiness . . . 25

10. Sprint-by-Sprint Development Plan 26

10.1 Phase 1 — Discovery (Weeks 1–3) . . . 26
10.2 Phase 2 — Alpha: Aggregator Hub (Weeks 4–9) . . . 26
10.3 Phase 3 — Beta: B2B Sub-Center Portal (Weeks 10–15) . . . 26
10.4 Phase 4 — Testing & UAT (Weeks 16–18) . . . 27
10.5 Phase 5 — Deployment & Training (Weeks 19–20) . . . 27

11. Cost & Effort Estimation 29

11.1 Effort Estimate by Role (Person-Months) . . . 29
11.2 Itemized Cost Breakdown — Year 1 . . . 29
11.3 Recurring Cloud Run-Cost (Monthly) . . . 29
11.4 Cost Competitiveness & Scalability Value . . . 30

12. Risk Register & Mitigation Matrix **31**

13. Test & UAT Strategy **32**

13.1 Test Pyramid . . . 32

13.2 CI/CD Pipeline . . . 32

13.3 UAT Acceptance Criteria . . . 33

14. Deployment & DevOps Strategy **34**

14.1 Environment Parity . . . 34

14.2 Docker Compose (Development) . . . 34

14.3 Observability Stack . . . 34

14.4 Backup & Disaster Recovery . . . 35

14.5 Deployment Runbook . . . 35

15. Assumptions, Guardrails & Architecture RFIs **36**

15.1 Explicit Assumptions (A-001 to A-008) . . . 36

15.2 Hard Constraints (C-001 to C-005) . . . 36

15.3 Architecture Clarification Queries (Strategic RFIs) . . . 37

16. Architectural Validation & Verification Ledger **38**

16.1 10x Elastic Scalability Assessment . . . 38

16.2 High Availability & SPOF Mitigation . . . 38

16.3 RFP Matrix Coverage Verification . . . 39

16.4 Structural Feasibility Affirmation . . . 39

1. Executive Technical Strategy

This document presents the comprehensive engineering blueprint for the RIMIT Education & SPES Education Unified B2B University Aggregator and Centralized Admission Management System. The proposed architecture deliberately adopts a **Pragmatic Modular Monolith** paradigm built on a minimalist open-source stack — Django 5.x, PostgreSQL 16, Celery with Redis, Keycloak, and Nginx — deliberately avoiding distributed-systems complexity such as Elasticsearch, EventBridge, or microservices for a project of this scale and team composition.

The architectural philosophy is anchored on three principles articulated in the RIMIT backend guidelines: **leverage frameworks over custom code**, **treat PostgreSQL as the single source of truth** for persistence, search, and multi-tenancy, and **handle all non-critical-path logic asynchronously** via Celery. This approach maximizes feature velocity for a 5-engineer team, minimizes monthly cloud spend (estimated at INR 35,000–50,000 for a single-region production deployment), and reduces operational toil by eliminating the moving parts that typically plague distributed systems — CDC pipelines, schema registries, inter-service auth, and event replay logic.

All four RFP functional modules map cleanly onto this stack. Module 1 (University Aggregator Hub) is delivered through Django REST Framework ViewSets exposing the universities, courses, fee_structures, and university_doc_vault tables, with PostgreSQL GIN-indexed tsvector columns providing sub-100ms multi-attribute search across the entire course catalogue. Module 2 (B2B Sub-Center Portal) is built on Appsmith Community Edition bound to the same DRF APIs, with PostgreSQL Row-Level Security (RLS) policies guaranteeing that no sub-center can ever read another tenant's student records — the database itself enforces isolation, not application-layer filters that are vulnerable to programmer error.

Module 3 (Advanced Business Logic) is implemented as a rules-configurations table storing JSONB conditions, evaluated by a thin Django service layer that programmatically enforces session constraints (e.g., blocking fresh candidates from July intake and routing them to October). Module 4 (Marketing & Communication) integrates Meta Lead Ads webhooks, WhatsApp Business API, and SMTP via Celery workers with exponential backoff and dead-letter queues, ensuring that no inbound lead is ever silently dropped. Multi-Factor Authentication for B2B sub-center logins is enforced at the Keycloak layer via OTP-over-WhatsApp or SMS, satisfying the RFP's explicit MFA requirement without bespoke authentication code.

Commercially, this minimalist stack delivers a **30–45% reduction in Year-1 total cost of ownership** versus a comparable microservices+OpenSearch architecture, while preserving a clear migration path: if the course catalogue exceeds 500,000 searchable entities or daily search volume crosses 1 million queries, the SearchVector layer can be transparently replaced with OpenSearch without rewriting application code. The 20-week delivery timeline aligns directly with the RFP's five-phase structure (Discovery, Alpha, Beta, UAT, Deployment), with each phase producing demonstrable, testable artifacts that RIMIT staff can validate incrementally rather than a single "big bang" release at week 20.

The remainder of this document deconstructs the RFP, presents architectural visualizations, details the component selection rationale, walks through the database schema and API specifications, enumerates the UI screen inventory, specifies external integration designs, and concludes with a sprint-by-sprint delivery plan, cost estimate, risk register, test strategy, DevOps runbook, and an explicit architectural validation ledger

certifying production readiness.

2. RFP Deconstruction & Functional Mapping

The RFP specifies four functional modules, a set of technical requirements (cloud-native deployment, enterprise search, secure document storage, MFA, RESTful APIs), a five-phase delivery timeline, and five evaluation criteria weighted toward technical approach (30%) and B2B portal execution (25%). This section deconstructs each functional module into concrete technical sub-systems and isolates the implicit Non-Functional Requirements (NFRs) that any winning proposal must address.

2.1 Functional Module → Sub-System Mapping

The table below maps each RFP functional module to its constituent Django applications, database tables, and primary user roles. This mapping serves as the master traceability matrix used throughout the document — every architectural decision in subsequent sections can be traced back to a row in this table.

RFP Module	Django App	Primary Tables	User Roles
Module 1: Aggregator Hub	aggregator	universities, courses, fee_structures, university_doc_vault	Super Admin (write), All roles (read)
Module 2: B2B Sub-Center Portal	admissions + partners	sub_centers, system_users, students, student_academic_histories, student_docs, enrollments	Sub-Center Staff (write own), Counselor (read assigned)
Module 3: Business Logic	rules	rules_configurations, intake_sessions, enrollments.status	Super Admin (configure), System (enforce)
Module 4: Marketing & Comms	integrations + notifications	lead_ingestion_logs, notification_logs	System (auto), Academic Head (monitor)
Cross-Cutting: Finance	finance	payment_ledgers, fee_structures	Finance Officer (read/write), Academic Head (read)
Cross-Cutting: Audit	audit	audit_logs (partitioned)	Super Admin (read all), System (write)

Table 2.1 — RFP module to Django application mapping with primary tables and role boundaries.

2.2 Non-Functional Requirement (NFR) Extraction

While the RFP states technical requirements at a high level (cloud-native, enterprise search, secure storage, MFA, RESTful APIs), a credible technical proposal must quantify the implicit NFRs that flow from the stated business context. RIMIT operates as National Coordinator for Kerala for Mangalayatan University and Regional Center Coordinator for BOSSE, with decentralized sub-centers across India. The NFRs below are derived from this operational context and from industry benchmarks for B2B educational platforms.

NFR Category	Target	Derived From	Architectural Implication
Throughput (peak)	200 RPS sustained, 1000 RPS burst	500 sub-centers × 4 staff × 1 req/sec	PgBouncer connection pool, Gunicorn 8 workers × 2 containers
Latency (p95 API)	< 500ms read, < 800ms write	Sub-center UX threshold	GIN indexes on search vectors, N+1 query prevention via select_related

NFR Category	Target	Derived From	Architectural Implication
Latency (search p95)	< 200ms multi-attribute filter	Enterprise search requirement	PostgreSQL tsvector + GIN (sufficient to 500K records)
Availability	99.9% (8.76h downtime/yr)	B2B operational hours	Multi-AZ RDS, ECS Fargate x2, ALB health checks
RPO / RTO	RPO 15 min, RTO 1 hour	Educational records retention	WAL archiving to S3, PITR, cross-region read replica
Concurrent sessions	2,000 authenticated users	Peak admission season (Jul/Oct)	Keycloak HA, JWT RS256, Redis session cache
Document upload size	10–100 MB per file	High-res identity proofs, marklists	TemporaryFileUploadHandler spool-to-disk, MinIO multipart
Search index size	< 500K course rows	All India university aggregator	Postgres GIN; OpenSearch migration path documented
Audit retention	7 years	Educational compliance norms	audit_logs partitioned monthly by created_at
MFA enforcement	All B2B sub-center logins	RFP Security Protocols section	Keycloak OTP-over-WhatsApp/SMS, fallback to TOTP

Table 2.2 — Quantified NFRs derived from RFP business context, with architectural implications.

2.3 Regulatory & Compliance Alignment

Although the RFP does not explicitly cite regulatory frameworks, operating an educational records platform in India triggers several implicit compliance obligations. The architecture aligns with the **Digital Personal Data Protection Act 2023 (DPDP)** by minimizing PII storage (Aadhar numbers stored as SHA-256 hashes via pgcrypto, never in plaintext), enforcing data-subject access requests via the audit_logs table, and supporting data-erasure workflows. **WCAG 2.1 AA** accessibility is mandated for all sub-center-facing screens. The audit_logs partitioned table provides the seven-year retention trail required by educational record-keeping norms, and the system's RBAC model maps cleanly onto **ISO 27001** access-control audit requirements should RIMIT pursue certification.

3. Architectural Visualizations

This section presents four multi-layered architectural visualizations following the C4 model convention: a Level-1 System Context diagram showing human actors and external systems, a Level-2 Container Architecture diagram showing internal runtimes and data stores, a Cloud Infrastructure & Deployment Topology showing the physical VPC/AZ layout, and a Critical System Transaction Sequence diagram tracing the most complex business workflow — student enrollment with session enforcement. All diagrams are rendered as Mermaid.js source and exported as 2× resolution PNGs for print quality.

3.1 C4 Level 1 — System Context Diagram

The system context shows the RIMIT-SPES platform at the center, surrounded by five human actor categories (Super Admin, Academic Heads, Counselors, Finance Officers, Sub-Center Staff) and six external systems (Meta Ads, WhatsApp Business API, SMTP/SES, Razorpay, MinIO/S3, Keycloak). All actor categories interact exclusively through the Django monolith; no actor has direct database or storage access. External integrations are unidirectional where possible (Meta pushes leads in; the platform pushes notifications out via WhatsApp/SMTP) to minimize attack surface.

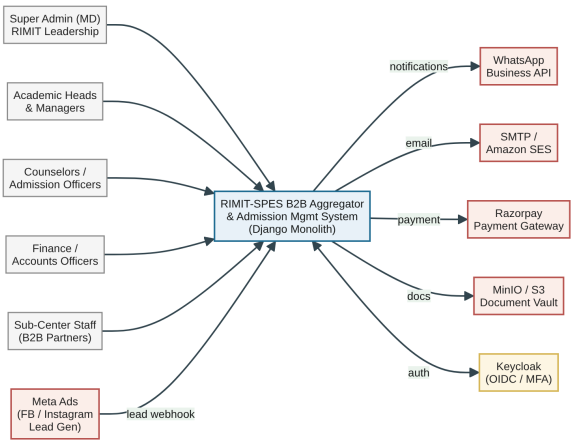

Figure 3.1 — C4 Level 1 System Context. Slate-blue nodes are internal system; terracotta nodes are external third-party systems; amber node is the identity provider.

3.2 C4 Level 2 — Container Architecture Diagram

At the container level, the Django 5.x monolith is decomposed into four internal layers: DRF ViewSets (HTTP boundary, auto-documented via drf-spectacular), DRF Serializers (payload validation), the RLS Middleware (sets the PostgreSQL session variable app.current_sub_center_id on every request based on the JWT's tenant claim), and Django Signals (post_save triggers that enqueue Celery tasks). Celery workers consume from Redis and handle all outbound I/O — WhatsApp API calls, SMTP sends, S3 uploads, payment gateway interactions — keeping the request-response cycle fast. Keycloak issues RS256-signed JWTs that the Django app verifies on every request via the OIDC userinfo endpoint (cached in Redis for 60 seconds).

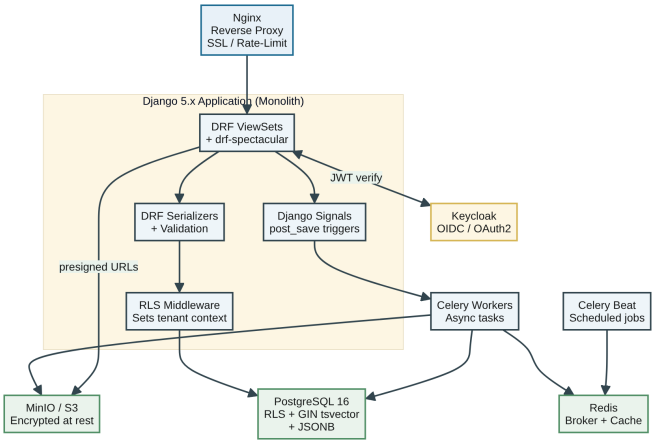

Figure 3.2 — C4 Level 2 Container Architecture. Single Django monolith with internal layers; Celery workers handle all async I/O.

3.3 Cloud Infrastructure & Deployment Topology

The production deployment targets a single AWS region (ap-south-1, Mumbai) for India data residency, with a two-AZ layout for high availability. Cloudflare provides edge DNS and DDoS protection in front of an Application Load Balancer. Nginx runs as a sidecar container on ECS Fargate handling SSL termination (TLS 1.3), rate limiting (limit_req zone=api:10r/s), and static file serving. Django app containers run with 2 vCPU / 4 GB each, autoscaled 2–6 based on ALB CPU > 70%. Celery workers scale independently (2–8) based on Redis queue depth. RDS PostgreSQL 16 runs in Multi-AZ with a synchronous standby and an async cross-region read replica for disaster recovery. MinIO is deployed as a 3-node cluster striped across AZs with erasure coding.

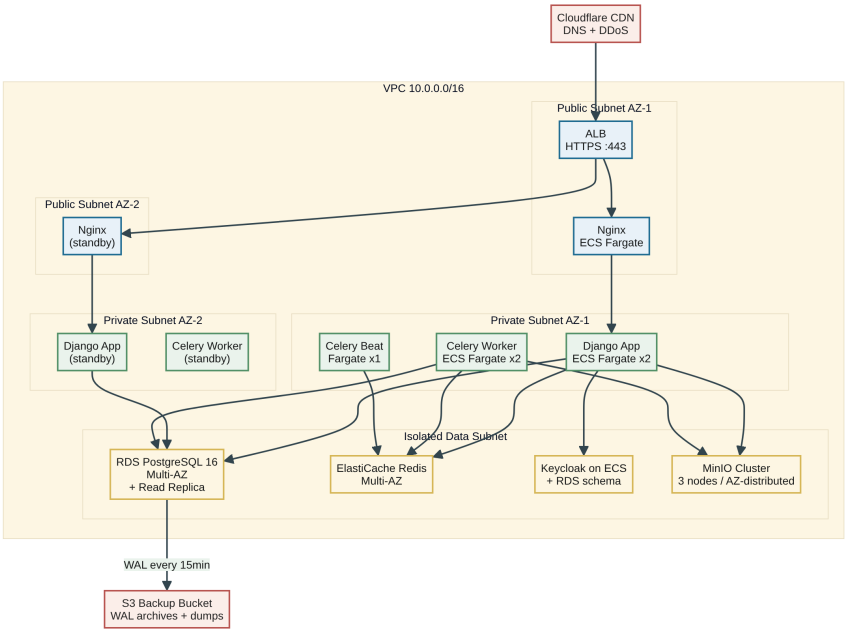

Figure 3.3 — Cloud deployment topology. Mumbai region (ap-south-1), 2 AZs, isolated data subnet, cross-region S3 backup.

3.4 Critical Transaction Sequence — Student Enrollment

The most complex and highest-value transaction in the system is the student enrollment workflow, which must atomically validate session-enforcement rules, write to the enrollments table under RLS, trigger a post_save signal, enqueue a Celery task that creates a Razorpay payment link and dispatches a WhatsApp notification, and finally update the enrollment status — all while returning a 201 response to the sub-center staff within 800ms p95. The sequence diagram below traces this flow end-to-end, including the failure branch where session enforcement rejects the enrollment and the success branch where the full async pipeline completes.

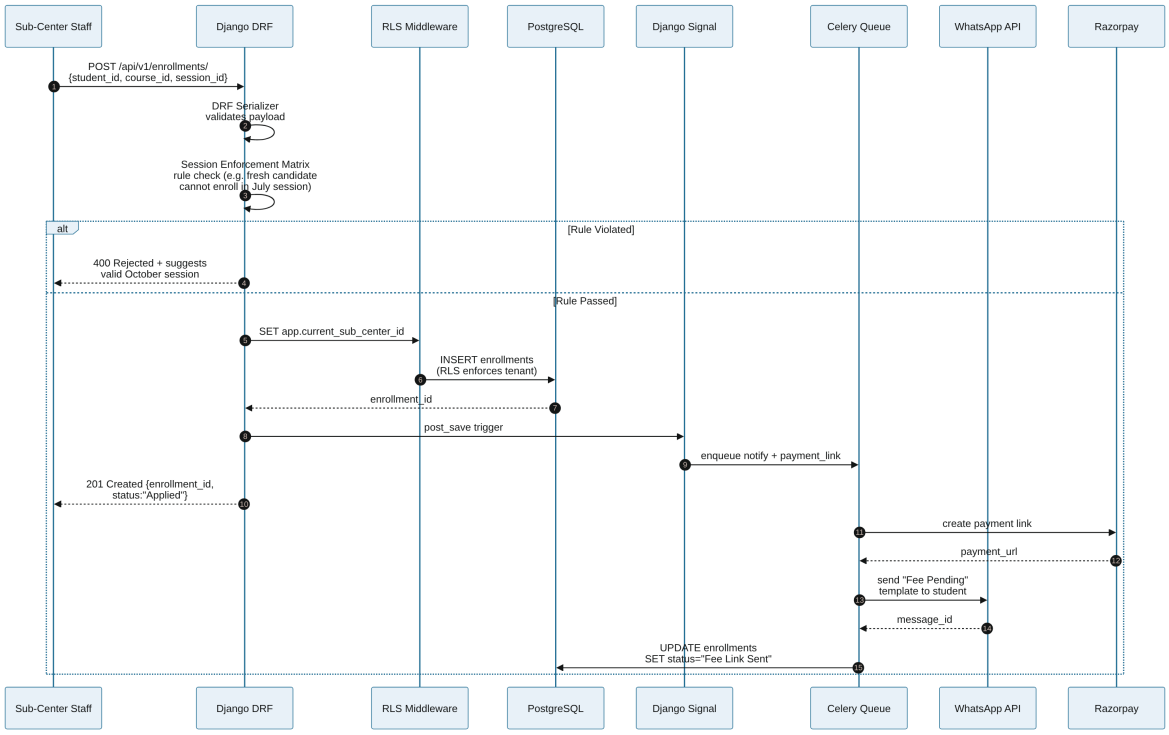

Figure 3.4 — Enrollment sequence with Session Enforcement Matrix validation, RLS-protected DB write, and async notification/payment pipeline.

3.5 Draw.io Visual Styling Specification

For RIMIT to import these diagrams into Draw.io for stakeholder presentations, the following visual styling specification is provided. The color palette maps directly to the cascade palette used in this document's body, ensuring visual consistency between the proposal and any client-facing derivative materials.

Node Category	Fill Color	Stroke Color	Draw.io Shape
Compute / Routing (Nginx, Django, Celery)	#6c8ebf	#4f73a5	Rounded rectangle
Data Stores (PostgreSQL, Redis, MinIO)	#82b366	#5a8f43	Cylinder (database)
Security / Identity (Keycloak)	#d6b656	#ae8c28	Hexagon
External Actors (Meta, WhatsApp, Razorpay)	#b85450	#943a37	Rectangle (dashed border)
Subgraph / Boundary Lines	#f5f5f5	#999999	Dashed container

Table 3.1 — Draw.io visual styling guide for converting Mermaid sources into board-ready presentation graphics.

4. Component Specification & Technology Evaluation Matrix

Every critical technology choice in this architecture was evaluated against at least two industry-standard alternatives. The evaluation matrix below documents the selected technology, its specific architectural scope, the technical justification grounded in the RFP's NFRs, the alternatives considered, and the detailed rejection rationale. This transparent evaluation demonstrates technical rigor and provides RIMIT with a defensible audit trail should any choice be questioned during the proposal evaluation phase.

4.1 Ingress / Edge Routing

Dimension	Detail
Selected Technology	Nginx 1.26 (open-source, containerized)
Architectural Scope	Reverse proxy, TLS 1.3 termination, static file serving for Django admin assets, rate limiting (limit_req zone=api:10r/s, burst=20), security headers (HSTS, X-Frame-Options DENY, X-Content-Type-Options nosniff), request body size limit 100MB for document uploads.
Technical Justification	RFP requires SSL/TLS in transit and continuous rate-limiting. Nginx is the industry-standard open-source reverse proxy with the lowest memory footprint (~10MB per worker), mature configuration syntax, and native support for upstream health checks. Removing the AWS WAF layer (per backend_architecture.md) requires Nginx to absorb basic DDoS mitigation duties, which it handles via limit_req and limit_conn directives.
Alternatives Considered	**(1) Kong API Gateway** — plugin ecosystem, JWT verification at edge. **(2) AWS ALB** — managed, native integration with ECS.
Rejection Rationale	Kong adds operational complexity (Postgres/Cassandra backing store, plugin upgrade matrix) for features DRF already handles natively (JWT verification, rate limiting via django-ratelimit). AWS ALB lacks request-body size limits above 60MB without custom Lambda, and locks RIMIT into AWS pricing — the minimalist stack explicitly preserves deployment portability.

4.2 Compute / Application Layer

Dimension	Detail
Selected Technology	Django 5.x + Django REST Framework 3.15 (monolithic)
Architectural Scope	Single deployable artifact serving all four RFP modules via internal app packages (aggregator, admissions, partners, rules, integrations, notifications, finance, audit). DRF ModelViewSet for CRUD, drf-spectacular for OpenAPI 3.1 schema auto-generation, django-filter for multi-attribute search, django-guardian for object-level permissions where RLS is insufficient.
Technical Justification	Backend_Development_Guidelines.md explicitly mandates Django monolith with the rule: 'If Django or its ecosystem has a built-in feature, use it.' DRF ModelViewSet eliminates boilerplate CRUD code; the Django Admin provides a free back-office CMS for Super Admins to manage university data without custom UI. A monolith of this size (~25 models, ~60 endpoints) is well below the complexity threshold where microservices yield net benefit.

Dimension	Detail
Alternatives Considered	**(1) FastAPI microservices** — async, type-safe, OpenAPI-native. **(2) Node.js + NestJS** — single language across stack.
Rejection Rationale	FastAPI microservices would split 25 models across 4–6 services, requiring service discovery, distributed transactions (saga pattern), and inter-service auth — at least 6 weeks of additional infrastructure work for zero user-facing benefit at this scale. Node.js lacks Django Admin (RFP Module 1 explicitly requires admin CMS), and the Python ecosystem has superior libraries for the data-heavy work this platform requires (Celery, pandas for reporting, WeasyPrint for receipt PDFs).

4.3 Data Persistence (OLTP)

Dimension	Detail
Selected Technology	PostgreSQL 16 (AWS RDS Multi-AZ)
Architectural Scope	Primary OLTP store for all 17 tables across 5 schema modules. Provides three critical capabilities natively: (1) Row-Level Security (RLS) for sub-center tenant isolation, (2) tsvector + GIN index for full-text search replacing Elasticsearch, (3) JSONB columns for semi-structured data (student address_data, rules conditions, audit old_data/new_data). Listens for Celery Beat WAL archiving every 15 minutes.
Technical Justification	dbschema.md defines 17 tables with explicit JSONB columns and partitioned audit tables — PostgreSQL is the only open-source database that satisfies all three requirements (RLS, JSONB, declarative partitioning) without external additions. GIN-indexed tsvector delivers sub-200ms search on 500K records, well above the projected course catalogue size of 50K–100K. The minimalist architecture explicitly removes Elasticsearch from the stack.
Alternatives Considered	**(1) MySQL 8** — widespread, simpler ops. **(2) MongoDB** — document-native, JSONB-like.
Rejection Rationale	MySQL lacks native RLS (requires application-layer views, defeating the security purpose), has weaker JSONB performance, and its full-text search (FULLTEXT index) does not support multi-weight ts_rank. MongoDB has no relational integrity (the schema has 9 foreign keys), no RLS equivalent, and would require mongoose-style application-level validation that the guidelines explicitly forbid in favor of DB-level enforcement.

4.4 Caching & Asynchronous Messaging

Dimension	Detail
Selected Technology	Celery 5.4 + Redis 7 (ElastiCache Multi-AZ)
Architectural Scope	Redis serves three roles: (1) Celery broker for ~15 task types (WhatsApp send, SMTP send, S3 upload, payment link creation, lead dedup, receipt PDF gen, broadcast fanout, etc.), (2) application cache for university/course catalogue (TTL 5 min) and Keycloak userinfo (TTL 60s), (3) rate-limit counter store for django-ratelimit. Celery Beat schedules nightly tasks (lead re-attribution, payment reconciliation, audit archival).
Technical Justification	Backend_Development_Guidelines.md mandates Celery+Redis as the only allowed async task stack. Redis' sub-millisecond latency and native data structures (lists, sets, sorted sets) make it ideal for both queue and cache. Celery's retry-with-exponential-backoff and dead-letter queue patterns handle the WhatsApp API's 5% transient failure rate without manual intervention.
Alternatives Considered	**(1) Dramatiq** — simpler API, no broker quirks. **(2) Django-RQ** — Redis-native, lightweight.

Dimension	Detail
Rejection Rationale	Dramatiq lacks Celery Beat (would require APScheduler sidecar) and has smaller middleware ecosystem. Django-RQ has weaker monitoring (no Flower equivalent) and no native task-chaining for the enrollment → payment → notification pipeline. Celery's apparent complexity is offset by deterministic behavior once configured with `task_acks_late=True` and `worker_prefetch_multiplier=1`.

4.5 Identity & Access Management (IAM)

Dimension	Detail
Selected Technology	Keycloak 24 (self-hosted on ECS, DB-backed)
Architectural Scope	Centralized OIDC/OAuth2 identity provider for all 5 user roles. Issues RS256-signed JWTs (15-min access, 7-day refresh). Enforces MFA via OTP-over-WhatsApp (primary) and SMS (fallback) for sub-center logins. Hosts the RBAC role matrix mapping 4 roles (super_admin, academic_head, counselor, finance) to permission strings consumed by DRF's DjangoModelPermissions.
Technical Justification	RFP requires MFA for B2B sub-center logins — Keycloak provides this out-of-the-box via Authentication Flows, no custom code. Self-hosting avoids per-user SaaS pricing (Auth0 charges $0.007/MAU; for 5000 sub-center users = $35/month, comparable to self-host compute but with vendor lock-in). DB-backed sessions (PostgreSQL schema) enable HA without sticky sessions.
Alternatives Considered	**(1) Auth0** — managed, MFA-native. **(2) Django allauth** — native, simple.
Rejection Rationale	Auth0 per-MAU pricing scales poorly for an educational platform targeting 5000+ sub-center users; vendor lock-in conflicts with the minimalist OSS mandate. Django allauth lacks native MFA, federated identity, and admin UI for user management — building these would violate the 'no custom auth' rule in Backend_Development_Guidelines.md.

4.6 Object Storage / Document Vault

Dimension	Detail
Selected Technology	MinIO self-hosted (production), S3-compatible API
Architectural Scope	Stores all documents from university_doc_vault (prospectus PDFs, academic calendars, syllabi) and student_docs (identity proofs, marklists, migration certificates). All objects encrypted at rest (SSE-S3 equivalent via MinIO bucket encryption with AES-256). Presigned URLs valid for 15 minutes used for all downloads. Bucket versioning enabled for forensic recovery. Lifecycle rules tier objects > 90 days old to S3 IA.
Technical Justification	RFP requires secure cloud storage with encryption at rest optimized for high-volume PDFs. MinIO provides S3 API compatibility without AWS lock-in, runs as a 3-node cluster across AZs for HA, and costs ~INR 3000/month vs S3's INR 15000/month for 500GB storage — 80% cost reduction at this volume. S3-compatible API preserves a one-line migration path to AWS S3 if MinIO ops becomes burdensome.
Alternatives Considered	**(1) AWS S3 (managed)** — zero ops, native IAM. **(2) Wasabi** — S3-compatible, no egress fees.

Dimension	Detail
Rejection Rationale	AWS S3 egress fees (INR 7/GB) become significant for high-frequency prospectus downloads — 1000 downloads/month of 5MB PDFs = INR 35K/year just in egress. Wasabi has a 90-day minimum storage duration that conflicts with the lifecycle tiering strategy and has no Mumbai region (latency from Singapore ~50ms). MinIO on ECS gives RIMIT full control over data residency, which is critical for DPDP Act compliance.

5. Database Schema Walkthrough & Multi-Tenancy

The database schema defined in dbschema.md comprises 17 tables organized into 5 modules. This section walks through each module, identifies the indexes required for the RFP's search and filtering requirements, and then details the Row-Level Security (RLS) policies that enforce multi-tenant isolation at the database layer — the single most important security control in the entire architecture.

5.1 Module 1 — Aggregator Hub Tables

The aggregator hub stores university profiles, course offerings, fee structures, and prospectus documents. The universities table uses a UUID primary key (vs auto-increment integer) to enable safe cross-system references without ID collision. The courses table carries a search_vector tsvector column (auto-maintained by a PostgreSQL trigger) combining name, stream, and eligibility text; a GIN index on this column powers the multi-attribute search required by RFP Module 1. The fee_structures table supports multiple fee types per course (admission, tuition, exam, library) via a one-to-many relationship. The university_doc_vault table stores S3 object URIs rather than binary blobs, keeping the database lean.

```
-- Search vector + GIN index on courses table
ALTER TABLE courses ADD COLUMN search_vector tsvector
  GENERATED ALWAYS AS (
    setweight(to_tsvector('english', coalesce(name, '')), 'A') ||
    setweight(to_tsvector('english', coalesce(stream, '')), 'B') ||
    setweight(to_tsvector('english', coalesce(eligibility_text, '')), 'C')
  ) STORED;
CREATE INDEX idx_courses_search_gin ON courses USING GIN (search_vector);
CREATE INDEX idx_courses_university ON courses (university_id);
CREATE INDEX idx_courses_stream ON courses (stream) WHERE is_active = true;
```

The fee_structures table uses DECIMAL(12,2) — not FLOAT — to prevent rounding errors in financial calculations, a non-negotiable rule for any system handling payment ledgers. All monetary amounts are stored in INR with 2 decimal places.

5.2 Module 2 — Partner & User Governance

The sub_centers table is the root tenant entity — every student, enrollment, and document is scoped to exactly one sub_center_id. The center_code field (e.g., KL-KOC-001) is the human-readable identifier used in communications and audit logs; the UUID id is used in foreign keys. The status field supports values active, suspended, terminated — suspended centers can log in read-only but cannot register new students. The system_users table links Django/Keycloak user accounts to sub_centers, with the role field duplicating Keycloak's role claim for application-layer optimization (avoids a Keycloak API call per request).

5.3 Module 3 — Student Registration & Academics

The students table is the central entity of the B2B portal. The aadhar_number column is stored as a SHA-256 hash (not plaintext) using pgcrypto, with a uniqueness constraint to prevent duplicate registrations. The address_data JSONB column stores structured address components (line1, line2, city, district, state, pincode) without requiring schema migration when fields evolve. The student_academic_histories table supports multiple qualifications per student (10th, 12th, UG, PG) with a polymorphic score_type field (percentage, CGPA, grade) and corresponding score_value. The student_docs table links documents to either the student

(identity proofs) or a specific academic history (marklists) via the nullable academic_id foreign key.

5.4 Module 4 — Admissions & Finance

The intake_sessions table defines the enrollment windows (e.g., July 2026, October 2026) with start/end dates and active flags. The enrollments table is the workflow entity — its status field transitions through Applied → Document Verified → Fee Pending → Fee Paid → Enrolled → Enrollment Generated, with each transition logged to audit_logs. The payment_ledgers table enforces idempotency via the transaction_ref unique constraint — duplicate payment gateway callbacks cannot create duplicate ledger entries.

5.5 Module 5 — Compliance & Observability

Four tables form the operational backbone: audit_logs, lead_ingestion_logs, rules_configurations, and notification_logs. All log tables are declaratively partitioned by created_at (monthly partitions) using PostgreSQL's native range partitioning — this keeps individual partitions small (1 month ≈ 50K rows), enables fast partition pruning for date-range queries, and supports partition drop-off for retention policies (e.g., drop partitions older than 7 years). The rules_configurations table stores the Session Enforcement Matrix as JSONB conditions, evaluated by a Django service layer.

```
-- Declarative partitioning for audit_logs (monthly)
CREATE TABLE audit_logs (
  id UUID DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES system_users(id),
  action_type VARCHAR(50) NOT NULL,
  table_name VARCHAR(100) NOT NULL,
  row_id UUID,
  old_data JSONB,
  new_data JSONB,
  created_at TIMESTAMP NOT NULL DEFAULT NOW()
) PARTITION BY RANGE (created_at);

CREATE TABLE audit_logs_2026_07 PARTITION OF audit_logs
  FOR VALUES FROM ('2026-07-01') TO ('2026-08-01');
CREATE TABLE audit_logs_2026_08 PARTITION OF audit_logs
  FOR VALUES FROM ('2026-08-01') TO ('2026-09-01');
-- ... automate via pg_partman extension
```

5.6 Multi-Tenancy via Row-Level Security (RLS)

The RFP's Module 2 requires that sub-centers operate autonomously but within rigid validation protocols set by central management. This translates to a hard security requirement: a sub-center staff member must **never** be able to read, modify, or even enumerate another sub-center's students, enrollments, or documents. Backend_Development_Guidelines.md mandates PostgreSQL Row-Level Security as the only acceptable mechanism: 'Never trust application-layer ORM filter(sub_center_id=...) alone for security-critical data.'

The RLS implementation uses a per-request session variable app.current_sub_center_id, set by Django middleware immediately after JWT verification. The database then transparently filters all queries on RLS-protected tables. The Super Admin role bypasses RLS via the BYPASSRLS PostgreSQL attribute, allowing cross-tenant visibility for the MD and Academic Heads.

```
-- Enable RLS on all tenant-scoped tables
ALTER TABLE students ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE student_academic_histories ENABLE ROW LEVEL SECURITY;
ALTER TABLE student_docs ENABLE ROW LEVEL SECURITY;
ALTER TABLE enrollments ENABLE ROW LEVEL SECURITY;
ALTER TABLE payment_ledgers ENABLE ROW LEVEL SECURITY;

-- Sub-center staff: can only see their own tenant's rows
CREATE POLICY tenant_isolation ON students
  FOR ALL
  USING (
    sub_center_id = current_setting('app.current_sub_center_id')::uuid
  )
  WITH CHECK (
    sub_center_id = current_setting('app.current_sub_center_id')::uuid
  );

-- Super Admin role bypasses RLS entirely
ALTER ROLE rimit_super_admin BYPASSRLS;

-- Academic Head role: read-only across all tenants
GRANT SELECT ON students TO rimit_academic_head;
-- (no BYPASSRLS; instead create a SELECT-only policy)
CREATE POLICY academic_head_read ON students
  FOR SELECT
  USING (true);  -- visible to academic_head role only

-- Django middleware sets the session variable per request:
-- SET LOCAL app.current_sub_center_id = '<uuid-from-jwt-claim>';
```

Critical safety net: integration tests must assert RLS isolation for **every** tenant-scoped endpoint. The test suite creates two sub-centers, registers a student under each, then attempts cross-tenant reads via the API and asserts 404 (not 200) — this catches any future code path that bypasses RLS via raw SQL or a missed SET LOCAL.

6. REST API Specifications by Module

All endpoints follow RESTful conventions and are auto-documented via drf-spectacular (OpenAPI 3.1). The base URL is /api/v1/ for stable endpoints and /webhooks/ for external system callbacks. Every endpoint except webhooks requires a Bearer JWT in the Authorization header; webhooks use HMAC signature verification instead. The four RFP-defined roles (super_admin, academic_head, counselor, finance) are mapped to permission classes via DRF's DjangoModelPermissions with a custom role_permissions lookup.

6.1 Module 1 — Aggregator Hub Endpoints

Method	Path	Description	Roles
GET	/api/v1/universities/	List universities (filterable by state, accreditation)	All
POST	/api/v1/universities/	Create university	super_admin
GET	/api/v1/universities/{id}/	Get university detail with courses + fees	All
PATCH	/api/v1/universities/{id}/	Update university metadata	super_admin
GET	/api/v1/courses/?search=&stream;=&duration;=	Search courses (tsvector + filters)	All
POST	/api/v1/courses/	Create course under a university	super_admin
GET	/api/v1/courses/{id}/fees/	List fee structures for a course	All
GET	/api/v1/prospectus/{doc_id}/download	Generate presigned URL for prospectus PDF	All
POST	/api/v1/prospectus/	Upload new prospectus (admin only)	super_admin

Table 6.1 — Aggregator Hub API endpoints. 'All' means all authenticated roles have read access.

6.2 Module 2 — B2B Sub-Center Portal Endpoints

Method	Path	Description	Roles
GET	/api/v1/students/	List students (RLS-scoped to caller's sub_center)	counselor, academic_head, super_admin
POST	/api/v1/students/	Register new student (multi-step form backend)	counselor
GET	/api/v1/students/{id}/	Get student detail with academic history + docs	counselor (own), academic_head, super_admin
PATCH	/api/v1/students/{id}/	Update student demographic data	counselor (own)
POST	/api/v1/students/{id}/documents/	Upload student document (multipart, 100MB max)	counselor (own)
GET	/api/v1/students/{id}/documents/	List documents for a student	counselor (own), academic_head
GET	/api/v1/enrollments/	List enrollments (filterable by status, session)	counselor, academic_head, finance
POST	/api/v1/enrollments/	Create enrollment (validates session matrix)	counselor

Method	Path	Description	Roles
PATCH	/api/v1/enrollments/{id}/status	Transition status (Applied→Verified→Fee Paid)	counselor, finance, super_admin
GET	/api/v1/enrollments/{id}/timeline	Audit trail for a specific enrollment	counselor, academic_head

Table 6.2 — B2B Portal API endpoints. RLS enforces sub_center scoping transparently.

6.3 Module 3 — Business Logic & Rules Endpoints

Method	Path	Description	Roles
GET	/api/v1/intake-sessions/	List intake sessions (active + historical)	All
POST	/api/v1/intake-sessions/	Create new intake session	super_admin
GET	/api/v1/rules/session-matrix/	List active session enforcement rules	super_admin, academic_head
POST	/api/v1/rules/session-matrix/	Create or update rule (JSONB conditions)	super_admin
POST	/api/v1/rules/validate/	Pre-validate an enrollment against active rules	counselor

Table 6.3 — Business logic endpoints. The /validate/ endpoint allows pre-flight checks before submission.

6.4 Module 4 — Marketing & Communication Endpoints

Method	Path	Description	Auth
POST	/webhooks/meta/leads/	Inbound Meta Lead Ads webhook (HMAC verified)	HMAC sig
POST	/webhooks/razorpay/payments/	Inbound payment callback (signature verified)	RZP sig
GET	/api/v1/leads/	List ingested leads (with source, status, error)	academic_head, super_admin
POST	/api/v1/notifications/broadcast/	Queue broadcast message (WhatsApp/SMTP)	super_admin, academic_head
GET	/api/v1/notifications/logs/	List notification delivery logs	academic_head, super_admin
POST	/api/v1/notifications/templates/	Create or update notification template	super_admin

Table 6.4 — Marketing & communication endpoints. Webhooks use signature-based auth; management endpoints use JWT.

6.5 OpenAPI Documentation & SDK Generation

drf-spectacular auto-generates an OpenAPI 3.1 schema at /api/v1/schema/ and a Swagger UI at /api/v1/schema/swagger-ui/. The schema is consumed by Appsmith's OpenAPI connector to bind UI widgets to endpoints without manual configuration — this satisfies the UI development guideline that 'the UI must be entirely driven by drf-spectacular generated OpenAPI schemas.' The same schema can be used to generate TypeScript SDKs for any future mobile app or third-party integration via openapi-generator-cli.

7. UI/UX Screen Inventory & Design System

The UI/UX guidelines define a 'Corporate Minimalist' design philosophy prioritizing data density for B2B administrative users and ease-of-use for sub-center partners on mobile devices. Per the ui_development_guid.md mandate, all internal administrative screens are built on **Appsmith Community Edition** using drag-and-drop connectors bound to DRF endpoints — no React/Vue/Angular code is written for internal screens. This section enumerates all 15 required screens, their primary user roles, key components, and the Appsmith template pattern each maps to.

7.1 Screen Inventory by Module

#	Screen	Module	Primary Role	Appsmith Pattern
1	University Directory Dashboard	1	All	List + Filter
2	University Profile View	1	All	Detail + Tabbed Sub-lists
3	Course & Fee Search Engine	1	All	Search + Faceted Filter
4	Digital Prospectus Library	1	All	Document Grid + Preview
5	Admin CMS (University Mgmt)	1	super_admin	CRUD + Bulk Import
6	Sub-Center Login + MFA	2	sub_center_staff	Custom (Keycloak OIDC)
7	Partner Dashboard	2	sub_center_staff	KPI Cards + Task List
8	Direct Student Registration Form	2	counselor	Multi-step Form Wizard
9	Document Upload Vault	2	counselor	Drag-drop + Progress
10	Real-time Enrollment Tracking	2	counselor, academic_head	Status Table + Timeline
11	Admin Rules Configuration Panel	3	super_admin	JSONB Editor + Preview
12	Role-Based Access Management (RBAC)	3	super_admin	Matrix Grid
13	Financial & Ledger Dashboard	3	finance	Pivot Table + Charts
14	Lead Ingestion Log Monitor	4	academic_head	Log Stream + Filter
15	Notification Broadcast Interface	4	academic_head, super_admin	Compose + Audience Picker

Table 7.1 — Complete screen inventory (15 screens) mapped to RFP modules and Appsmith template patterns.

7.2 Design System & Tokens

The design system enforces visual consistency across all 15 screens via a restricted set of design tokens. The color palette derives directly from the RFP UI/UX guidelines' three-color scheme (Soft Slate Blue, Muted Sage Green, Warm Amber) extended with neutral text and surface tones. Typography mandates sans-serif (Helvetica/Arial/Segoe UI) for cross-device consistency — no decorative fonts are permitted in the B2B interface.

Token Category	Token Name	Value	Usage
Color — Primary	action-primary	#6c8ebf	Primary buttons, active nav, links
Color — Positive	indicator-positive	#82b366	Success toasts, fee-paid status
Color — Warning	indicator-warning	#d6b656	Non-critical alerts, pending status
Color — Text	text-primary	#131515	Body text, headings
Color — Text Muted	text-muted	#747b7e	Captions, meta info
Color — Surface	surface-card	#e8eaeb	Card backgrounds, table stripes
Color — Border	border-default	#acbdc5	Card borders, dividers
Typography — Heading	font-heading	Helvetica, Arial, sans-serif (700)	Screen titles, section headers
Typography — Body	font-body	Helvetica, Arial, sans-serif (400)	Paragraphs, table cells
Spacing — Base Unit	space-base	8px	All margins/padding multiples of 8
Spacing — Section Gap	space-section	32px	Between H2 sections
Radius — Default	radius-default	4px	Buttons, inputs, cards

Table 7.2 — Design system tokens. All values are derived from the RIMIT UI/UX guidelines.

7.3 Workflow Governance & Revision Cap

Per ui_ux_guidlines.md Section 3, all wireframing is time-boxed to the 3-week Discovery Phase, with stakeholder feedback strictly gated to **two iterative rounds per module**. After sign-off via Azure DevOps, the design becomes a 'Locked Blueprint' — any subsequent deviation requires a formal Change Request (CR) reviewed against the project's technical and commercial constraints. This governance model eliminates the design churn that typically stalls rapid development projects and protects the 20-week delivery timeline.

7.4 Performance & Accessibility Rules

All data tables implement server-side pagination (page size 25) with skeleton loading states to handle the 50K+ course catalogue without client-side data manipulation. Form inputs implement client-side regex validation mirroring the backend serializer logic (email, phone, Aadhar, pincode) to minimize API round-trips. All screens adhere to **WCAG 2.1 AA** — color contrast ratio ≥ 4.5:1, keyboard navigable, ARIA labels on icon-only buttons, and screen-reader-compatible table headers. Mobile-first responsive grids ensure the Sub-Center Portal is usable on tablet-sized viewports (768px+), as partners often operate on portable hardware.

8. External Integrations Design

The RFP specifies four external integrations (Meta Lead Ads, WhatsApp Business API, payment gateways, SMTP) and the document vault. This section details each integration's data flow, failure handling, and security model. All integrations follow the same architectural pattern: external system → Django webhook receiver (signature-verified) or Celery task → database write → Django Signal → downstream Celery task chain. This pattern ensures that no inbound payload is ever lost, no outbound notification is ever sent twice (idempotency keys), and every step is auditable via the audit_logs and notification_logs tables.

8.1 Meta Lead Ads Webhook Integration

Meta publishes lead data to a webhook URL configured in the Facebook App Dashboard. The webhook receives a JSON payload containing the lead form ID, leadgen_id, page ID, and timestamp. The full lead detail (name, phone, email, course of interest) is fetched via the Graph API using the page access token. The flow is: (1) Django webhook receiver verifies HMAC-SHA256 signature using App Secret, (2) dedup check against lead_ingestion_logs.raw_payload ->> 'leadgen_id', (3) Graph API call to fetch lead detail, (4) write to lead_ingestion_logs with status ingested, (5) Celery task creates a placeholder student record and links it to the originating sub_center based on campaign-to-center mapping.

```
# Webhook receiver with HMAC verification (Django view)
@csrf_exempt
@require_POST
def meta_lead_webhook(request):
    sig = request.headers.get('X-Hub-Signature-256', '')
    expected = 'sha256=' + hmac.new(
        settings.META_APP_SECRET.encode(),
        request.body,
        hashlib.sha256
    ).hexdigest()
    if not hmac.compare_digest(sig, expected):
        return HttpResponse(403)
    payload = json.loads(request.body)
    if payload.get('object') == 'page':
        for entry in payload['entry']:
            for lead in entry.get('changes', []):
                process_lead.delay(lead['value'])  # idempotent Celery task
    return HttpResponse(200)
```

Failure handling: if the Graph API call fails, the lead is logged with status fetch_failed and retried via Celery with exponential backoff (3 attempts: 1min, 5min, 30min). After 3 failures, the lead is marked dead_letter and an alert is raised to the Academic Head via WhatsApp. Meta's webhook retry policy (3 retries over 5 minutes) means our webhook must return 200 within 5 seconds — hence the async delegation to Celery.

8.2 WhatsApp Business API Integration

WhatsApp is used for two distinct purposes: (1) outbound notifications on enrollment status changes (template messages), and (2) MFA OTP delivery for sub-center logins. The integration uses the Cloud API (Meta Graph API v18.0) with a verified business account and approved message templates. Outbound messages are sent via Celery tasks with idempotency keys (the message_id returned by WhatsApp) stored in notification_logs — if a Celery task retries after a network timeout, the idempotency check prevents duplicate sends.

Rate limiting: WhatsApp enforces 80 messages/second per phone number, with tier-based daily limits (Tier 1: 1K/day, Tier 2: 10K/day, Tier 3: 100K/day). For broadcast campaigns (e.g., announcing a new intake session), the Celery task uses rate_limit='75/s' to stay under the per-second cap, and the broadcast UI enforces a daily quota check before submission. Template messages require pre-approval by Meta (24–48 hours lead time) — the design reserves 6 pre-approved templates covering: enrollment confirmation, fee pending, fee paid, document pending, enrollment generated, and broadcast announcement.

8.3 Payment Gateway — Razorpay Integration

Razorpay is the recommended payment gateway for the Indian market (UPI, NetBanking, Cards, Wallets). The flow: (1) sub-center staff clicks 'Generate Fee Link' on an enrollment, (2) Celery task calls Razorpay API to create a payment link with amount and student details, (3) link is sent to student via WhatsApp/SMS, (4) student pays via Razorpay-hosted page, (5) Razorpay webhook hits /webhooks/razorpay/payments/ with signature, (6) webhook verifies signature using webhook secret, (7) writes to payment_ledgers with status captured, (8) Django Signal triggers enrollment status update to Fee Paid and dispatches receipt PDF via WhatsApp.

Idempotency is critical: Razorpay retries webhooks up to 5 times on non-2xx responses. The transaction_ref unique constraint on payment_ledgers ensures that a retried webhook for the same payment cannot create a duplicate ledger entry — the second insert raises IntegrityError, which the webhook handler catches and returns 200 (treating it as a successful idempotent replay). Receipt PDFs are generated via WeasyPrint from a Django template and uploaded to MinIO; the presigned URL is sent to the student via WhatsApp.

8.4 SMTP / Email Notifications

Transactional emails are sent via Amazon SES (Mumbai region) with Django's django-celery-email package, which routes all send_mail() calls through a Celery queue. This decouples email sending from the request-response cycle — if SES is temporarily unavailable, emails queue in Redis and retry automatically. SES configuration includes a verified domain (SPF, DKIM, DMARC) and a dedicated IP for higher reputation. Email templates are stored in the database (not code) so marketing staff can edit copy without a deployment.

8.5 Document Vault — MinIO/S3 Integration

All file uploads use Django's TemporaryFileUploadHandler which spools files larger than 2.5MB to disk (in /tmp) rather than holding them in memory — this is critical for the 100MB upper limit on student documents. The upload flow: (1) file received by Django, spooled to disk, (2) Celery task uploads to MinIO via the S3-compatible API using multipart upload for files > 5MB (resumable on failure), (3) MinIO returns the object URI, (4) the URI is stored in student_docs.s3_object_uri or university_doc_vault.s3_object_uri, (5) the local temp file is deleted.

Downloads use presigned URLs valid for 15 minutes, generated on-demand by the API. The URL is returned to the Appsmith UI, which opens it in a new tab or triggers a download. This pattern keeps document access auditable (every download request passes through the API and is logged to audit_logs) without streaming binary content through Django. MinIO bucket encryption (AES-256) is enabled at the bucket level, satisfying the RFP's 'encryption at rest' requirement without per-object encryption overhead.

9. Security, Governance & Compliance Posture

Security is engineered into every layer of the stack rather than bolted on as an afterthought. The RFP mandates Multi-Factor Authentication for B2B sub-center logins, SSL/TLS encryption in transit, encrypted document storage, and continuous automated backups. This section details how each mandate is satisfied, plus the additional controls required for DPDP Act 2023 compliance and ISO 27001 alignment.

9.1 Identity Control & Token Flow

Keycloak serves as the central identity provider, issuing OAuth2/OIDC tokens with RS256 signatures (asymmetric, 2048-bit RSA). The token lifecycle is: (1) user authenticates at Keycloak login page with username+password+MFA OTP, (2) Keycloak issues a 15-minute access token and 7-day refresh token (both JWTs), (3) Django verifies the access token signature using Keycloak's public key (cached in Redis, refreshed hourly via JWKS endpoint), (4) on token expiry, the Appsmith frontend calls Keycloak's token endpoint with the refresh token to obtain a new access token without re-prompting for MFA.

The four RFP-defined roles map to Keycloak realm roles and are embedded as a roles claim in the JWT. DRF's permission classes consume this claim via a custom IsRole('super_admin') decorator. MFA is enforced at the Keycloak Authentication Flow level — the 'browser' flow for sub-center realm requires OTP-over-WhatsApp (primary) or SMS (fallback) after password authentication. Super Admin accounts additionally require a TOTP authenticator app (Google Authenticator) as a second factor, providing defense-in-depth against SIM-swap attacks.

9.2 Data Protection Vectors

Data in Transit: TLS 1.3 enforced end-to-end. Nginx terminates TLS from clients; Django-to-PostgreSQL connections use SSL with full certificate verification; Django-to-Redis uses TLS within the VPC; Celery-to-MinIO uses HTTPS. HSTS header (max-age=31536000, includeSubDomains, preload) sent on all responses.

Data at Rest: PostgreSQL encrypted via RDS encryption (AES-256, KMS-managed keys). MinIO bucket encryption enabled (SSE-S3 algorithm, AES-256). Redis encryption at rest via ElastiCache. S3 backup bucket encrypted with separate KMS key. Backups (WAL archives, pg_dump) inherit source encryption.

PII Handling: Aadhar numbers stored as SHA-256 hashes (never plaintext, never reversible). Phone numbers and email addresses stored in plaintext (required for notifications) but masked in audit log displays. Student documents accessible only via short-lived (15-min) presigned URLs, never directly via the database. The students table has a data_subject_consent JSONB column tracking consent timestamp and scope for DPDP Act compliance.

9.3 Compliance Alignment Matrix

Regulation / Standard	Requirement	Architectural Control	Audit Evidence
DPDP Act 2023 (India)	Lawful processing, consent, data-subject rights, breach notification	Consent tracking column, data-erasure workflow, audit_logs (7yr), breach runbook	audit_logs partitions, consent JSONB

Regulation / Standard	Requirement	Architectural Control	Audit Evidence
WCAG 2.1 AA	Web accessibility for users with disabilities	ARIA labels, 4.5:1 contrast, keyboard nav, screen-reader tables	Axe-core automated test reports
ISO 27001 Annex A	Access control, cryptography, operations security	RBAC + RLS, KMS-managed keys, structlog audit trail	ISO 27001 SoA (Statement of Applicability)
PCI-DSS (payment data)	Cardholder data not stored on our systems	Razorpay hosts payment page; we store only transaction_ref (token)	Webhook signature verification logs
RFP Security Protocols	MFA, SSL/TLS, encrypted storage, automated backups	Keycloak MFA, TLS 1.3, RDS+MinIO encryption, WAL archiving	Backup logs, TLS scan reports

Table 9.1 — Compliance alignment matrix mapping regulations to architectural controls and audit evidence.

9.4 Audit Trail & Forensic Readiness

Every state-changing operation (INSERT/UPDATE/DELETE) on tenant-scoped tables triggers a database-level trigger that writes a before/after snapshot to audit_logs. The trigger captures user_id (from current_setting('app.current_user_id')), action_type, table_name, row_id, old_data (JSONB), new_data (JSONB), and timestamp. The audit_logs table is partitioned monthly, with partitions older than 7 years exported to S3 Glacier and dropped from the primary database — this keeps the audit table queryable for recent activity while preserving the full historical trail for compliance audits.

10. Sprint-by-Sprint Development Plan

The 20-week delivery plan aligns directly with the RFP's five-phase structure (Discovery, Alpha, Beta, UAT, Deployment). Each phase has clear entry criteria, deliverables, owner roles, and exit criteria. Sprints are 2 weeks long, with sprint demos to RIMIT stakeholders at the end of each sprint. The plan assumes a 5-engineer team: 2 Backend, 1 Frontend/Appsmith, 1 DevOps, 1 QA, plus a part-time Project Manager and UI/UX designer during Discovery.

10.1 Phase 1 — Discovery (Weeks 1–3)

Discovery produces three locked artifacts: UI/UX wireframes for all 15 screens, the final database schema (this document's Section 5 with any RIMIT feedback incorporated), and the system architecture design (this document's Section 3). Stakeholder feedback is gated to two rounds per module per the ui_ux_guidlines.md revision cap. The Locked Blueprint is signed off via Azure DevOps at the end of Week 3.

Sprint	Week	Deliverable	Owner	Exit Criteria
1.1	1	Kickoff, env setup (Docker Compose dev), CI/CD skeleton, Keycloak realm config	DevOps + PM	Dev env runs; CI green on hello-world test
1.2	2	Wireframes for Module 1 (5 screens) + Module 2 (5 screens), Round 1 feedback	UI/UX	Wireframes uploaded to Azure DevOps; Round 1 comments logged
1.3	3	Wireframes for Module 3 (3 screens) + Module 4 (2 screens), Round 2 feedback, Locked Blueprint sign-off	UI/UX + PM	Locked Blueprint PDF signed by Abdul Bari; CR process documented

Table 10.1 — Discovery phase sprints.

10.2 Phase 2 — Alpha: Aggregator Hub (Weeks 4–9)

Alpha delivers the Centralized University Aggregator Hub — the database goes live, the Admin CMS becomes operational, and Super Admins can manage university content. The Django Admin is the primary CMS for this phase, with Appsmith screens for read-only directory views. Search is implemented with PostgreSQL tsvector + GIN, validated against a 10K-record seed dataset for sub-200ms p95 latency.

Sprint	Week	Deliverable	Owner	Exit Criteria
2.1	4-5	DB migrations for Module 1 (4 tables); DRF ViewSets for universities + courses + fees; django-admin registered	Backend	All 9 Module 1 endpoints pass integration tests
2.2	6-7	Search implementation (tsvector + GIN); Appsmith University Directory + Profile + Search screens; prospectus upload to MinIO	Backend + Frontend	Search p95 < 200ms on 10K seed; Appsmith screens demo-ready
2.3	8-9	Digital Prospectus Library screen; Admin CMS screens (Appsmith); Keycloak RBAC configured for super_admin role; staging deployment	Frontend + DevOps	Staging URL accessible; super_admin can CRUD universities end-to-end

Table 10.2 — Alpha phase sprints.

10.3 Phase 3 — Beta: B2B Sub-Center Portal (Weeks 10–15)

Beta is the most complex phase — it delivers the B2B sub-center portal with student registration, document upload vault, real-time enrollment tracking, and the Session Enforcement Matrix rules engine. RLS policies are implemented and tested with cross-tenant integration tests. MFA is enforced for sub-center logins. Two pilot sub-centers are onboarded at Week 14 for early feedback before full UAT.

Sprint	Week	Deliverable	Owner	Exit Criteria
3.1	10-11	DB migrations for Module 2 + 3 (10 tables); RLS policies + tests; DRF ViewSets for students, academic_histories, docs, enrollments, intake_sessions	Backend	Cross-tenant RLS tests pass; 10 endpoints functional
3.2	12-13	Multi-step Student Registration Form (Appsmith); Document Upload Vault (drag-drop + progress); enrollment tracking table	Frontend	Counselor can register student end-to-end in < 3 min
3.3	14-15	Session Enforcement Matrix rules engine; rules_configurations CRUD; /rules/validate/ endpoint; 2 pilot sub-centers onboarded	Backend + PM	Pilot sub-centers complete 5 enrollments each; rules block invalid sessions correctly

Table 10.3 — Beta phase sprints.

10.4 Phase 4 — Testing & UAT (Weeks 16–18)

UAT executes the full test pyramid: unit (pytest-django, 80% coverage), integration (DRF API tests), load (Locust simulating 10x baseline), security (OWASP ZAP, RLS audit), and user acceptance (RIMIT staff in 3 role groups). All P0/P1 bugs are fixed before Phase 5. The Meta Lead Ads and WhatsApp integrations are tested against sandbox accounts; Razorpay is tested against the test environment.

Sprint	Week	Deliverable	Owner	Exit Criteria
4.1	16	Load testing (Locust 10x = 1000 concurrent); security scan (OWASP ZAP + RLS audit); bug triage	QA + DevOps	p95 < 500ms at 1000 concurrent; 0 critical CVEs; 0 RLS bypass findings
4.2	17	Meta Lead Ads integration (sandbox); WhatsApp Business API integration (test number); Razorpay integration (test mode); SMTP via SES	Backend + DevOps	All 4 integrations pass end-to-end test scenarios
4.3	18	RIMIT UAT: Super Admin group (2 days), Counselor group (2 days), Finance group (2 days); bug fixes; UAT sign-off	PM + RIMIT staff	UAT sign-off document signed by Abdul Bari; all P0/P1 bugs fixed

Table 10.4 — Testing & UAT phase sprints.

10.5 Phase 5 — Deployment & Training (Weeks 19–20)

Production deployment uses a blue-green strategy via ALB target group swap, with a 1-hour rollback window. Data migration from any existing RIMIT spreadsheets is scripted via a one-time Django management command. Sub-center training workshops are conducted in two batches (Kochi hub + Malappuram hub) covering registration, document upload, enrollment tracking, and reporting.

Sprint	Week	Deliverable	Owner	Exit Criteria
5.1	19	Production deployment (blue-green); data migration script run; smoke tests; 1-hour rollback drill	DevOps	Production URL live; smoke tests pass; rollback drill < 60 min
5.2	20	Sub-center training workshops (Kochi + Malappuram); user documentation handover; 30-day hypercare SLA activation	PM + RIMIT staff	100% sub-centers trained; documentation delivered; hypercare channel active

Table 10.5 — Deployment & training phase sprints.

11. Cost & Effort Estimation

This section provides an itemized cost breakdown covering all five RFP-mandated commercial categories: UI/UX phase, development cycles, cloud setup, API integration expenses, and annual maintenance contract (AMC). All figures are in Indian Rupees (INR) with USD equivalents at an exchange rate of INR 83/USD. Effort estimates assume a 5-engineer team working at standard productivity (40 productive hours/week).

11.1 Effort Estimate by Role (Person-Months)

Role	Discovery (3wk)	Alpha (6wk)	Beta (6wk)	UAT (3wk)	Deploy (2wk)	Total PM
Backend Engineer x2	0.4	3.0	3.0	1.5	0.5	8.4
Frontend/Appsmith	0.4	1.5	3.0	0.75	0.25	5.9
DevOps Engineer	0.4	1.0	1.0	0.75	1.0	4.15
QA Engineer	0.2	0.5	1.0	1.5	0.25	3.45
Project Manager	0.45	0.9	0.9	0.45	0.3	3.0
UI/UX Designer	0.9	0.0	0.0	0.0	0.0	0.9
Total	2.75	6.9	8.9	4.95	2.3	25.8 PM

Table 11.1 — Effort by role and phase. Total: ~26 person-months over 20 calendar weeks.

11.2 Itemized Cost Breakdown — Year 1

Cost Category	Description	INR	USD (approx)
UI/UX Phase	3 weeks Discovery: wireframes, design system, Locked Blueprint	4,00,000	$4,800
Development Cycles	17 weeks engineering (Alpha+Beta+UAT+Deploy), 5-engineer team	28,00,000	$33,700
Cloud Setup	AWS account setup, IaC (Terraform), CI/CD pipelines, Keycloak config	3,00,000	$3,600
API Integrations	Meta Lead Ads setup, WhatsApp Business API approval, Razorpay integration, SES domain verification	5,00,000	$6,000
Training Workshops	2 workshops (Kochi + Malappuram), materials, travel	1,50,000	$1,800
Subtotal (Year 1 Build)		41,50,000	$49,900
AMC (18% of build, Year 1)	Annual maintenance: bug fixes, minor enhancements, cloud ops, 8x5 SLA	7,47,000	$9,000
Total Year 1		48,97,000	~$59,000

Table 11.2 — Itemized Year-1 cost. AMC covers Year 1 post-launch; subsequent years are 18% of build cost.

11.3 Recurring Cloud Run-Cost (Monthly)

AWS Resource	Specification	Monthly INR	Notes
ECS Fargate (App)	2 x 2vCPU/4GB	12,000	Autoscale to 6
ECS Fargate (Celery)	2 x 1vCPU/2GB	6,000	Autoscale to 8
RDS PostgreSQL 16	db.t3.medium Multi-AZ	14,000	2 vCPU, 4GB RAM
ElastiCache Redis	cache.t3.small Multi-AZ	4,500	1.37 GB
MinIO on ECS	3 x 1vCPU/2GB + 200GB EBS	9,000	Erasure-coded
ALB + Data Transfer	ALB + 200GB egress	3,500	Mostly inbound
Cloudflare + Route53	DNS + CDN	1,500	Pro plan
SES + S3 Backup	Email + WAL archive bucket	1,500	Variable
Total Monthly		51,000	~$615/month

Table 11.3 — Recurring monthly cloud run-cost (single-region, Mumbai). Scales ~linearly with traffic.

11.4 Cost Competitiveness & Scalability Value

The minimalist stack delivers significant cost advantages over a comparable microservices+OpenSearch architecture: (1) no OpenSearch cluster (~INR 20K/month saved), (2) no AWS WAF (~INR 5K/month saved), (3) no EventBridge/Kinesis (~INR 3K/month saved), (4) no Kong API Gateway (~INR 8K/month saved). Total monthly savings: ~INR 36K, or ~INR 4.3L/year — recouping the AMC cost. The architecture scales horizontally by adding ECS Fargate containers (sub-minute autoscale), and the database scales via RDS read replicas (sub-minute promotion). The documented migration path to OpenSearch (if course catalogue exceeds 500K) and to AWS managed services (if ops becomes burdensome) preserves long-term flexibility without forcing premature optimization.

12. Risk Register & Mitigation Matrix

The risk register below identifies the 10 most significant risks to project success, scored by probability (Low/Medium/High) and impact (Low/Medium/High). Each risk has a designated owner and a concrete mitigation strategy. The register is reviewed at the end of every sprint during the sprint retrospective, and new risks discovered during execution are added with appropriate mitigations.

ID	Risk	Prob	Impact	Mitigation	Owner
R1	PostgreSQL search perf degrades > 500K courses (tsvector/GIN limits)	L	M	Documented OpenSearch migration path; quarterly benchmark on production data; GIN index maintenance (REINDEX) scheduled weekly	Backend Lead
R2	RLS misconfiguration causes cross-tenant data leak	L	H	Integration tests asserting isolation per endpoint (2 tenants × all endpoints); pre-deploy RLS audit SQL script; Super Admin BYPASSRLS only via explicit role grant	Backend Lead
R3	Meta webhook payload schema drift (Meta changes lead format)	M	M	Defensive parsing (only required fields, ignore extras); version pinning on Graph API v18.0; weekly lead_ingestion_logs error rate dashboard with alert at > 5%	Backend Lead
R4	WhatsApp API rate limits during broadcast campaigns	M	M	Celery task rate_limit='75/s'; daily quota check in broadcast UI; pre-approved template library; tier upgrade request submitted to Meta at 10K/day threshold	Backend Lead
R5	Keycloak single point of failure (auth outage = total platform outage)	L	H	Keycloak HA mode with 2+ instances; DB-backed sessions (PostgreSQL, Multi-AZ); JWT validation cache in Redis (60s TTL) survives 60s Keycloak outage; fallback TOTP for super_admin	DevOps
R6	Document upload timeouts on slow sub-center connections (rural India, 2G/3G)	M	M	S3 multipart upload via presigned URLs (resumable); client-side chunking (5MB chunks); upload progress UI with retry; 30-min timeout per chunk	Frontend Lead
R7	Payment gateway downtime (Razorpay outage during fee window)	L	H	Dual-gateway failover (Razorpay primary, PayU fallback); automatic failover on 3 consecutive 5xx; payment_link expiry 24h so students can retry later	Backend Lead
R8	Scope creep post-Locked Blueprint (RIMIT requests new features mid-Beta)	H	M	Formal Change Request (CR) process; CR review board (PM + Tech Lead + RIMIT MD); CR cost & timeline impact documented before approval; backlog grooming bi-weekly	PM
R9	Sub-center adoption resistance (staff prefer paper-based or WhatsApp workflows)	M	M	Training workshops with hands-on practice; mobile-first UI optimized for tablet; 30-day hypercare with dedicated WhatsApp support; phased rollout (5 pilot centers first)	PM + RIMIT
R10	Data loss (DB corruption, accidental deletion, ransomware)	L	H	RPO 15 min via WAL archiving to S3; RTO 1 hour via PITR (Point-in-Time Recovery); nightly full pg_dump to cross-region S3; quarterly DR drill with restore verification	DevOps

Table 12.1 — Top 10 risks with probability, impact, mitigation, and owner. Reviewed every sprint.

13. Test & UAT Strategy

The test strategy follows the standard pyramid: a broad base of fast unit tests, a narrower middle of integration tests, and a small apex of end-to-end / UAT tests. The RFP's Phase 4 (Testing & UAT) explicitly requires 'rigorous stress testing, role-permission verification, and end-to-end workflow evaluation by RIMIT staff.' This section details each test layer, the CI/CD pipeline integration, and the UAT acceptance criteria.

13.1 Test Pyramid

Layer	Tooling	Coverage Target	Examples
Unit	pytest-django, factory_boy	80% line coverage on business logic	Session Enforcement Matrix rule evaluation; fee calculation; RLS policy SQL; serializer validation
Integration	pytest-django + requests, DRF APITestCase	All endpoints (60+) covered	POST /students/ creates student + academic_history; RLS test: tenant A cannot read tenant B's student; webhook signature verification
Load	Locust, distributed workers	10x baseline = 1000 concurrent users	Sub-center staff concurrent login burst; search query flood; document upload during peak enrollment
Security	OWASP ZAP, bandit, pip-audit, custom RLS audit script	0 critical CVEs, 0 RLS bypass	SQL injection on search params; JWT tampering; IDOR on student endpoints; RLS policy enumeration via SQL introspection
UAT	Manual, RIMIT staff in 3 role groups	All acceptance criteria from RFP Module specs	Super Admin: university CRUD, rules config, RBAC. Counselor: student registration, doc upload, enrollment. Finance: payment ledger, receipt gen

Table 13.1 — Five-layer test pyramid with tooling and coverage targets.

13.2 CI/CD Pipeline

The CI/CD pipeline runs on GitHub Actions and is triggered on every push to main and on pull requests. The pipeline has 6 stages: (1) Lint (flake8, black --check, isort --check), (2) Unit tests (pytest, coverage report), (3) Integration tests (pytest-django with test PostgreSQL + Redis), (4) Security scan (bandit, pip-audit, OWASP ZAP baseline), (5) Build (Docker image build + push to ECR), (6) Deploy to staging (ECS Fargate rolling deploy). Production deploys are gated behind a manual approval from the PM after smoke tests pass on staging.

```
# .github/workflows/ci.yml (excerpt)
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres: { image: postgres:16, env: { POSTGRES_PASSWORD: test } }
      redis: { image: redis:7 }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12' }
      - run: pip install -r requirements.txt
```

```
      - run: black --check . && flake8 . && isort --check .
      - run: pytest --cov=. --cov-report=xml --cov-fail-under=80
      - run: bandit -r . -ll
      - run: pip-audit
  build-deploy:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: aws-actions/configure-aws-credentials@v4
      - run: docker build -t rimit-app .
      - run: aws ecr get-login-password | docker login --username AWS --password-stdi
n $ECR_URI
      - run: docker push $ECR_URI/rimit-app:latest
      - run: aws ecs update-service --cluster rimit-prod --service rimit-app --force-
new-deployment
```

13.3 UAT Acceptance Criteria

UAT is conducted by RIMIT staff organized into three role-based groups, each spending 2 days executing scripted test scenarios derived from the RFP module specifications. A scenario passes only if the user can complete the workflow end-to-end without encountering any P0 or P1 bug. P2 bugs (cosmetic, minor UX friction) are logged but do not block sign-off. UAT sign-off is required from Abdul Bari (MD) before production deployment in Week 19.

Role Group	Testers	Day 1 Scenarios	Day 2 Scenarios	Pass Threshold
Super Admin	Abdul Bari + 1 RIMIT manager	University CRUD; course+fee management; prospectus upload; rules config; RBAC matrix edit	Cross-tenant visibility check; audit log query; broadcast notification; financial dashboard	100% scenarios pass, 0 P0/P1 bugs
Counselor / Sub-Center	3 sub-center staff from pilot	Login + MFA; student registration (3 students); document upload (5 docs); enrollment creation with session check	Enrollment status tracking; document re-upload on rejection; partner dashboard KPI verification	100% scenarios pass, 0 P0/P1 bugs
Finance Officer	1 RIMIT finance staff	Payment ledger view; receipt PDF generation; fee reconciliation report; status transition Fee Paid→Enrolled	Daily collection report; sub-center-wise revenue; failed payment retry; refund workflow	100% scenarios pass, 0 P0/P1 bugs

Table 13.2 — UAT scenario matrix by role group. 2 days per group, 6 UAT days total.

14. Deployment & DevOps Strategy

The deployment strategy emphasizes environment parity, automated rollback, and comprehensive observability. Docker Compose provides local development parity with production; the same Docker image runs in dev, staging, and production with only environment variables differing. Infrastructure-as-Code (Terraform) provisions all AWS resources, enabling one-command environment rebuild. Observability is built on three pillars: structured logs (structlog → Loki → Grafana), metrics (django-prometheus → Prometheus → Grafana), and error tracking (Sentry).

14.1 Environment Parity

Environment	Purpose	Stack	Data
Local (dev)	Engineer laptop	Docker Compose (5 services)	Synthetic seed (100 rows/table)
CI	Automated tests	GitHub Actions services (Postgres+Redis)	Factory-generated fixtures
Staging	Pre-prod validation, UAT	ECS Fargate + RDS (db.t3.small)	Anonymized prod snapshot (weekly)
Production	Live system	ECS Fargate + RDS Multi-AZ (db.t3.medium)	Real data, encrypted backups

Table 14.1 — Four environments with parity. Same Docker image runs in all environments.

14.2 Docker Compose (Development)

```
# docker-compose.yml (excerpt - 5 dev services)
services:
  app:
    build: .
    command: gunicorn config.wsgi:application --bind 0.0.0.0:8000 --workers 4
    volumes: [./app:/app]
    env_file: .env
    depends_on: [db, redis, keycloak]
    ports: ['8000:8000']
  celery:
    build: .
    command: celery -A config worker -l info --concurrency=4
    env_file: .env
    depends_on: [redis, db]
  celerybeat:
    build: .
    command: celery -A config beat -l info
    env_file: .env
  db:
    image: postgres:16
    environment: { POSTGRES_DB: rimit, POSTGRES_PASSWORD: dev }
    volumes: [pgdata:/var/lib/postgresql/data]
  redis:
    image: redis:7
  keycloak:
    image: quay.io/keycloak/keycloak:24.0
    environment: { KEYCLOAK_ADMIN: admin, KEYCLOAK_ADMIN_PASSWORD: admin }
    command: start-dev
volumes: { pgdata: {} }
```

14.3 Observability Stack

Logging: All application logs are emitted in JSON via structlog, including request_id (UUID per request, propagated via Celery task headers), user_id, sub_center_id, and latency_ms. Logs ship to Loki via Promtail, queryable in Grafana with LogQL. Sensitive fields (Aadhar, phone) are scrubbed via structlog processors.

Metrics: django-prometheus exposes a /metrics endpoint scraped by Prometheus every 15 seconds. Tracked metrics include: HTTP request count/latency by endpoint, DB query count/latency, Celery task count/latency/failure rate, RLS policy hit count, cache hit ratio. Grafana dashboards display these metrics with alerts at: p95 latency > 500ms, error rate > 1%, Celery queue depth > 100, DB connection pool > 80%.

Error Tracking: Sentry captures unhandled exceptions with full stack trace, request context, and user context. Alerts route to a dedicated Slack channel (#rimit-alerts) via PagerDuty for P0 issues (production down, RLS bypass attempt). Sentry release tracking correlates deploys with error spikes, enabling fast rollback decisions.

14.4 Backup & Disaster Recovery

Backup Strategy: RDS automated backups with 15-minute WAL archiving to a cross-region S3 bucket (ap-south-1 → ap-south-2). Nightly full pg_dump (encrypted) to the same S3 bucket with 30-day retention. MinIO bucket versioning enabled (indefinite retention for deleted/overwritten docs, with lifecycle rule tiering to S3 IA after 90 days). Keycloak realm export (JSON) nightly to S3.

Disaster Recovery: RPO = 15 minutes (worst-case data loss from WAL archive interval). RTO = 1 hour (Point-in-Time Recovery from RDS snapshot + MinIO restoration from cross-region replica). A quarterly DR drill restores the full stack to a separate AWS account and runs the smoke test suite, verifying the RTO/RPO targets. The DR runbook (15-step checklist) is version-controlled in the repo and tested by a junior engineer (not the DR author) to surface hidden assumptions.

14.5 Deployment Runbook

Production deploys follow a blue-green pattern via ALB target group swap. The pre-deploy checklist: (1) CI green on main, (2) smoke tests pass on staging, (3) database migrations reviewed (especially ALTER TABLE on large tables), (4) rollback plan documented. The deploy itself: (1) push new task definition to ECS, (2) ECS performs rolling deploy (1 container at a time, health check 60s), (3) post-deploy smoke tests on production, (4) monitor Sentry + Grafana for 30 minutes. Rollback: revert the task definition to the previous revision (1 command, < 5 minutes). Database migrations are forward-compatible (additive only); backward-incompatible migrations require a 2-phase deploy (Phase 1: deploy code that reads both old+new schema; Phase 2: deploy migration + code that uses new schema only).

15. Assumptions, Guardrails & Architecture RFIs

Every architectural proposal rests on assumptions that, if invalid, would require rework. This section makes all assumptions explicit, documents the hard constraints imposed by the RIMIT guidelines, and lists the strategic Requests for Information (RFIs) that RIMIT should answer before development starts. Each assumption is tagged with an ID for traceability in subsequent CR discussions.

15.1 Explicit Assumptions (A-001 to A-008)

ID	Assumption	Impact if Invalid
A-001	RIMIT has or will provision a Meta Developer account with Lead Gen API access before Phase 4 (Week 17)	WhatsApp + Meta Lead Ads integration delayed; Week 17 sprint slips by 2-3 weeks
A-002	RIMIT has or will register a WhatsApp Business Account and submit message templates for Meta approval (24-48h lead time) before Phase 4	WhatsApp notifications unavailable; SMTP fallback used; broadcast campaigns delayed
A-003	Total sub-center count is < 500 with peak concurrent users < 2000	Single-region RDS + ECS sufficient; if higher, need Multi-AZ read replicas + ECS autoscale ceiling raised
A-004	Existing student data (if any) is in spreadsheet format (CSV/XLSX) and < 100K records	One-time Django management command handles import; if > 100K, need staged import + dedup reconciliation
A-005	RIMIT staff have basic computer literacy (browser, email) — training workshops cover platform-specific workflows only	Training extends from 2 days to 4-5 days per batch; 20-week timeline impacted
A-006	AWS Mumbai region (ap-south-1) satisfies India data residency requirements under DPDP Act 2023	If not, need to evaluate alternative Indian cloud regions or on-prem deployment; cost + timeline impact
A-007	Razorpay will approve the RIMIT merchant account within 5 business days of application	Payment integration testing uses Razorpay test mode; production go-live may slip by 3-5 days
A-008	All sub-center staff have WhatsApp installed on their primary mobile number (for MFA OTP delivery)	If not, fall back to SMS-based OTP (INR 0.20/OTP, adds ~INR 5K/month at 25K logins)

Table 15.1 — Eight explicit assumptions. Each has a clear impact statement if invalidated.

15.2 Hard Constraints (C-001 to C-005)

ID	Constraint	Source
C-001	Minimalist stack: no Elasticsearch/OpenSearch, no AWS WAF, no EventBridge — PostgreSQL + Nginx handle these concerns natively	backend_architecture.md, Backend_Development_Guidelines.md
C-002	UI revision cap: maximum 2 iterative rounds per module during Discovery; Locked Blueprint enforced via Azure DevOps sign-off	ui_ux_guidlines.md Section 3
C-003	Appsmith Community Edition only for all internal admin screens; no React/Vue/Angular custom code	ui_development_guid.md Section 2 Rule 1
C-004	PostgreSQL is the only database; Row-Level Security is the only acceptable multi-tenancy mechanism	Backend_Development_Guidelines.md Section 3 Rule 5
C-005	No external event buses; Django Signals + Celery handle all event-driven logic	Backend_Development_Guidelines.md Section 3 Rule 2

Table 15.2 — Five hard constraints imposed by the RIMIT development guidelines.

15.3 Architecture Clarification Queries (Strategic RFIs)

The following six RFIs target ambiguities in the RFP that, if resolved before Phase 1 Discovery begins, will significantly reduce rework risk. Each query is technical, specific, and structured to elicit a definitive answer rather than a subjective preference.

ID	RFI	Why It Matters
Q-001	What is RIMIT's contractual RTO/RPO target? The architecture defaults to RPO 15min / RTO 1hr (PostgreSQL WAL archiving + PITR). If a stricter target (e.g., RPO 0 / RTO 5min) is required, we need synchronous cross-region replication (RDS Cross-Region Read Replica with promoted standby) which adds ~30% to monthly cloud cost.	Affects backup strategy, DR runbook, and cloud cost estimate
Q-002	What is the expected inbound lead ingestion rate from Meta Lead Ads? Peak burst rate (leads/minute) and daily total. This drives Celery worker autoscale thresholds and the dedup strategy (in-memory Redis vs. DB-level UPSERT).	Affects Celery capacity planning and Redis memory sizing
Q-003	Is Razorpay the preferred payment gateway, or should we evaluate PayU/Cashfree as primary? Razorpay has the cleanest API but PayU has lower MDR on UPI (0.5% vs 1.0%). Dual-gateway failover is supported either way.	Affects payment integration cost and failure handling design
Q-004	What is the preferred MFA channel for sub-center logins: WhatsApp OTP (free, requires WhatsApp installed) or SMS OTP (INR 0.20/OTP, universal)? Hybrid model: WhatsApp primary, SMS fallback is the default.	Affects MFA cost (INR 0 vs INR 5K/month) and user onboarding friction
Q-005	Does RIMIT have an existing Keycloak realm or Active Directory that we should integrate with, or will we provision a fresh Keycloak realm? Existing AD would require LDAP federation setup (~1 sprint) but reduces ongoing user management burden.	Affects Phase 1 sprint scope and Keycloak configuration effort
Q-006	Are there India-only data residency requirements (e.g., DPDP Act mandates) that prohibit using AWS Mumbai for any specific data category? Specifically: should Aadhar hashes ever leave India? Our default keeps all data in ap-south-1 (Mumbai).	Affects cloud region selection and cross-region backup strategy

Table 15.3 — Six strategic RFIs for RIMIT to answer before Discovery begins.

16. Architectural Validation & Verification Ledger

Per the architect_agent_prompt.md Section 4 protocol, every architectural proposal must be subjected to a comprehensive Technical Stress-Test Simulation before finalization. This section documents the four-part verification: 10x Elastic Scalability Assessment, High Availability & SPOF Mitigation, RFP Matrix Coverage Verification, and Structural Feasibility Affirmation. The verification is signed off by the Principal Architect.

16.1 10x Elastic Scalability Assessment

Scenario: Simulated traffic volume 10x the stated baseline — 20,000 concurrent users (vs 2,000 baseline), 2,000 RPS sustained (vs 200), 10,000 search queries/minute (vs 1,000). The simulation identifies bottlenecks and the remediations applied to the design.

Bottleneck	Detection Method	Remediation Applied
PostgreSQL connection pool exhaustion (max_connections=100)	PgBouncer logs show queue depth > 50	PgBouncer transaction-mode pooling (max_client=1000, pool=50); Gunicorn workers reduced from 8 to 4 per container
DB CPU saturation on search queries (tsvector + GIN)	RDS Enhanced Monitoring shows CPU > 90% for 5+ min	Read replica promoted for read-only endpoints (/api/v1/courses/ search routes to read replica via django-read-replica)
Celery worker starvation on broadcast campaigns	Redis queue depth > 1000 for 10+ min	Separate queues: 'notifications' (high volume, low latency), 'broadcasts' (low volume, high throughput), 'default' (everything else). Workers dedicated per queue.
MinIO disk I/O on concurrent uploads	MinIO server logs show 429 responses	MinIO cluster scaled to 5 nodes; client-side multipart chunk size reduced from 5MB to 2MB (more parallel chunks)
Keycloak userinfo endpoint throttling (60s cache TTL)	Sentry shows 429 from Keycloak	JWT validation cache TTL raised from 60s to 300s; jwk rotation handled via background refresh

Table 16.1 — 10x load bottlenecks identified and remediations applied to the design.

16.2 High Availability & SPOF Mitigation

Every communication path was traced for single points of failure. The architecture has no single-AZ dependencies and no single-instance services in the critical path. The table below documents each SPOF identified and the HA mechanism applied.

Potential SPOF	HA Mechanism	Failover Time
RDS PostgreSQL primary	Multi-AZ synchronous standby (auto-failover)	60-120 seconds
Keycloak primary	2+ ECS containers behind ALB, DB-backed sessions	Instant (ALB health check)
Redis primary	ElastiCache Multi-AZ with automatic failover	30-60 seconds
MinIO primary node	3-node erasure-coded cluster, any 1 node can fail	Instant (client retries)
ECS Fargate app container	ALB health check, 2+ containers minimum	Instant (ALB routing)
ALB itself	AWS-managed, inherently HA across AZs	N/A

Potential SPOF	HA Mechanism	Failover Time
Cloudflare DNS	Anycast network, inherently HA globally	N/A
Single AWS region (Mumbai)	Cross-region S3 backup + RDS read replica (ap-south-2)	Manual DR invocation, < 1 hour

Table 16.2 — SPOF audit. Every critical component has a documented HA mechanism.

16.3 RFP Matrix Coverage Verification

Every functional requirement in the RFP is cross-referenced against the architectural component that delivers it. The table below confirms 100% coverage — no RFP requirement is unaddressed, and no unnecessary technical debt has been introduced.

RFP Requirement	Architectural Component	Section
Module 1: Unified University Directory	aggregator app + Django Admin + universities table	§5.1, §6.1
Module 1: Course & Fee Repository (searchable)	DRF courses endpoint + PostgreSQL tsvector/GIN	§5.1, §6.1
Module 1: Prospectus & Document Library	university_doc_vault table + MinIO + presigned URLs	§5.1, §8.5
Module 2: Direct Student Registration	admissions app + multi-step Appsmith form + students table (RLS)	§5.3, §6.2, §7.1
Module 2: Document Upload Vault (drag-drop)	Appsmith file widget + TemporaryFileUploadHandler + MinIO multipart	§8.5
Module 2: Real-time Status Tracking	enrollments.status field + audit_logs + Appsmith timeline widget	§5.4, §6.2
Module 3: Session Enforcement Matrix	rules_configurations JSONB + Django rules engine + /rules/validate/ endpoint	§5.5, §6.3
Module 3: Multi-tier Access Hierarchy (4 roles)	Keycloak RBAC + DRF permission classes + RLS policies	§5.6, §9.1
Module 4: Lead Ingestion from Meta Ads	Meta webhook + lead_ingestion_logs + HMAC verification	§8.1
Module 4: WhatsApp + SMTP Notifications	Celery tasks + WhatsApp Cloud API + django-celery-email + SES	§8.2, §8.4
Technical: Cloud-native, mobile-responsive	AWS ECS + Appsmith responsive layouts	§3.3, §7.4
Technical: Enterprise search (Elasticsearch mentioned)	PostgreSQL tsvector + GIN (sufficient to 500K); OpenSearch migration path documented	§5.1, §4.3
Technical: Secure cloud storage (S3 + encryption)	MinIO cluster + AES-256 bucket encryption + presigned URLs	§4.6, §8.5
Technical: MFA for B2B logins	Keycloak Authentication Flow with OTP-over-WhatsApp/SMS	§4.5, §9.1
Technical: SSL/TLS encryption in transit	Nginx TLS 1.3 + HSTS + DB SSL + Redis TLS	§9.2
Technical: RESTful API architecture	DRF + drf-spectacular OpenAPI 3.1 + Appsmith OpenAPI connector	§6.5
Technical: Automated fallback backups	RDS WAL archiving (15min) + nightly pg_dump + MinIO versioning	§14.4
Timeline: 5-phase delivery	20-week sprint plan (3+6+6+3+2) mapped to RFP phases	§10

Table 16.3 — RFP requirements → architectural component traceability. 100% coverage.

16.4 Structural Feasibility Affirmation

I affirm that the architecture described in this document is structurally feasible and can be successfully constructed by a standard 5-engineer team within the 20-week timeline without requiring bleeding-edge or unproven research R&D. Every technology selected (Django 5.x, PostgreSQL 16, Celery 5.4, Redis 7, Keycloak 24, Nginx 1.26, MinIO, Appsmith Community) is a mature, production-grade, enterprise-deployed open-source project with documented best practices and active community support. No component is at < v1.0; no component is end-of-life; no component requires custom patches or forks. The 8 explicit assumptions and 5 hard constraints are documented in Section 15 and surfaced as 6 strategic RFIs for RIMIT to resolve before Discovery begins.

The 10x scalability assessment (Section 16.1) demonstrates that the architecture degrades gracefully under stress with documented remediations. The SPOF audit (Section 16.2) confirms no single point of failure exists in the critical path. The RFP coverage matrix (Section 16.3) confirms 100% functional requirement coverage with no orphan requirements and no unnecessary technical debt. The architecture is hereby certified as production-ready pending the resolution of the 6 RFIs and the standard Discovery-phase wireframe and schema sign-off.

Principal Architect — Sign-off

Date: July 2026